

Discrete Mathematics (MATH 208)

Fall 2022

Friday, 10/14/2022

Homework 7: Chapters 16, 17, 18

|             |
|-------------|
| Score: / 10 |
|-------------|

Name (Print): \_\_\_\_\_

---

## Chapter 16 - Mathematical Induction

4 pts

1. Prove that for every integer  $n \geq 1$ ,

$$1 \cdot 2 + 2 \cdot 3 + 3 \cdot 4 + \cdots + n(n+1) = \frac{n(n+1)(n+2)}{3}.$$

|       |
|-------|
| 4 pts |
|-------|

2. Prove by induction: For  $n \geq 2$ ,

$$\left(1 - \frac{1}{4}\right) \left(1 - \frac{1}{9}\right) \left(1 - \frac{1}{16}\right) \cdots \left(1 - \frac{1}{n^2}\right) = \frac{n+1}{2n}.$$

## Chapter 17 - Algorithms

5 (bonus)

1. Roman numerals are represented by seven different symbols: I, V, X, L, C, D and M.

For example, 2 is written as *II* in Roman numeral, just two ones added together. The number 12 is written as *XII*, which is simply  $X + II$ . The number 27 is written as *XXVII*, which is  $XX + V + II$ .

| Symbol | Value |
|--------|-------|
| I      | 1     |
| V      | 5     |
| X      | 10    |
| L      | 50    |
| C      | 100   |
| D      | 500   |
| M      | 1000  |

Roman numerals are usually written largest to smallest from left to right. However, the numeral for four is not *IIII*. Instead, the number four is written as *IV*. Because the one is before the five we subtract it making four. The same principle applies to the number nine, which is written as *IX*. There are six instances where subtraction is used:

- I (1) can be placed before V (5) and X (10) to make 4 and 9.
- X (10) can be placed before L (50) and C (100) to make 40 and 90.
- C (100) can be placed before D (500) and M (1000) to make 400 and 900.

Design an algorithm that will convert a roman numeral to an integer.

## Chapter 18 - Algorithm Efficiency

2 pts

1. Here's an old Yiddish joke:

*Shlemiel gets a job as a street painter, painting the dotted lines down the middle of the road. On the first day he takes a can of paint out to the road and finishes 300 yards of the road. "That's pretty good!" says his boss, "you're a fast worker!" and pays him a kopeck. The next day Shlemiel only gets 150 yards done. "Well, that's not nearly as good as yesterday, but you're still a fast worker. 150 yards is respectable," and pays him a kopeck. The next day Shlemiel paints 30 yards of the road. "Only 30!" shouts his boss. "That's unacceptable! On the first day you did ten times that much work! What's going on?" "I can't help it," says Shlemiel. "Every day I get farther and farther away from the paint can!"*

People often refer to certain algorithms in reference to this joke, a tradition attributed to Joel Spolsky (one of the programmers responsible for the popular programming question-and-answer site Stack Overflow), as a way to describe how they might perform well on small test cases, but become hopelessly inefficient at scale due to needlessly redundant operations being performed. In practice, algorithms with this type of inefficiency are not always as easy to spot as Shlemiel's needlessly redundant walking-back-and-forth. Explain the Schlemiel-the-painter inefficiency in the following algorithm:

---

**Algorithm:** Add two Numbers

---

**Input:** two positive integers  $n, m$

**Output:**  $n + m$

```
1: sum  $\leftarrow n$ 
2:  $k \leftarrow 1$ 
3: while  $k \leq m$  do
4:   sum  $\leftarrow$  sum + 1
5:    $k \leftarrow k + 1$ 
6: end while
7: output sum
```

---